SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and

stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases;	Handles most cases;	Handles basic cases only	Incomplete or inefficient

		optimized approach	reasonable efficiency		

Algorithm: Network Performance Analysis
START

INPUT total_data_received = 8000 bits
INPUT time = 4 seconds
INPUT total_packets_sent = 200
INPUT total_packets_received = 180

// Validate input values
IF time <= 0 THEN
  DISPLAY "Error: Time must be greater than zero."
  STOP
END IF

IF total_packets_sent <= 0 THEN
  DISPLAY "Error: Total packets sent must be greater than zero."
  STOP
END IF

// Calculate throughput
throughput = total_data_received / time

// Calculate packet loss
packet_loss = total_packets_sent - total_packets_received
packet_loss_percentage = (packet_loss / total_packets_sent) * 100

// Display calculated results
DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
DISPLAY "Total Data Received: ", total_data_received, " bits"
DISPLAY "Time: ", time, " seconds"
DISPLAY "Packets Sent: ", total_packets_sent
DISPLAY "Packets Received: ", total_packets_received
DISPLAY "Throughput: ", throughput, " bps"
DISPLAY "Packet Loss: ", packet_loss_percentage, "%"

```
// Classify network
IF throughput > 1500 AND packet_loss_percentage < 10 THEN
   network_status = "Efficient"
ELSE
   network_status = "Inefficient"
END IF

DISPLAY "Network Classification: ", network_status

END
```

Calculation:
Throughput = 8000 / 4 = 2000 bps
Packet loss = 200 − 180 = 20 packets
Packet loss percentage = (20 / 200) × 100 = 10%
Since throughput is greater than 1500 bps, but packet loss is not less than 10%, the network is classified as Inefficient.
Final Summary: The network achieves 2000 bps throughput with 10% packet loss, so according to the given conditions, its overall performance is Inefficient.

Algorithm: Monitor Bandwidth and Detect Overloaded Links

```
START

INPUT number_of_links

CREATE array bandwidth_usage[number_of_links]

WHILE TRUE DO

   DISPLAY "----- BANDWIDTH MONITORING -----"

   FOR i = 0 TO number_of_links - 1 DO

      INPUT bandwidth_usage[i]
```

```
        DISPLAY "Link ", i + 1, " Utilization: ",
              bandwidth_usage[i], "%"

        IF bandwidth_usage[i] > 80 THEN
           DISPLAY "WARNING: Link ", i + 1,
                 " is overloaded."
           DISPLAY "Recommendation: Switch traffic to backup link."
        ELSE
           DISPLAY "Link ", i + 1,
                 " is operating normally."
        END IF

     END FOR

     WAIT for a fixed monitoring interval

END WHILE

END
```

The algorithm stores the utilization of every network link in an array. The FOR loop checks each link during every monitoring cycle. When utilization exceeds 80%, the algorithm identifies the link as overloaded and recommends moving traffic to a backup link.

This supports efficient bandwidth management because the administrator can identify heavily utilized links before they become severely congested. Redirecting traffic to available backup links distributes the network load, reduces packet delays and packet loss, and improves overall network performance. Continuous monitoring also allows the system to respond dynamically when traffic conditions change.

Algorithm: Dynamic Best Path Selection Based on Link Quality

```
START

INPUT number_of_links
```

```
INPUT number_of_paths

CREATE arrays:
   bandwidth[number_of_links]
   delay[number_of_links]
   utilization[number_of_links]
   link_status[number_of_links]
   quality_score[number_of_links]

WHILE TRUE DO

   DISPLAY "----- NETWORK MONITORING -----"

   // Collect current link information
   FOR i = 0 TO number_of_links - 1 DO

      INPUT bandwidth[i]
      INPUT delay[i]
      INPUT utilization[i]
      INPUT link_status[i]

      IF link_status[i] = "FAILED" THEN
         quality_score[i] = 0

         DISPLAY "Link ", i + 1, " has failed."
         DISPLAY "Alternate path will be selected."

      ELSE IF utilization[i] > 80 THEN
         quality_score[i] = 0

         DISPLAY "Link ", i + 1, " is overloaded."
         DISPLAY "Avoid this link."

      ELSE
         // Calculate link quality score
         quality_score[i] =
            (0.5 * bandwidth[i])
```

```
            - (0.3 * delay[i])
            - (0.2 * utilization[i])

    END IF

END FOR

// Find the path having the highest overall quality
best_score = -INFINITY
best_path = NONE

FOR each path from headquarters to every branch DO

    path_score = 0
    path_available = TRUE

    FOR each link in the current path DO

        IF link_status[link] = "FAILED"
          OR utilization[link] > 80 THEN

            path_available = FALSE
            BREAK

        END IF

        path_score = path_score + quality_score[link]

    END FOR

    IF path_available = TRUE AND path_score > best_score THEN
        best_score = path_score
        best_path = current path
    END IF

END FOR
```

```
    IF best_path = NONE THEN
        DISPLAY "ALERT: No reliable path available."
        DISPLAY "Administrator attention required."

    ELSE
        DISPLAY "Best communication path: ", best_path
        DISPLAY "Path Quality Score: ", best_score
    END IF

    // Alert administrator about abnormal conditions
    FOR i = 0 TO number_of_links - 1 DO

        IF link_status[i] = "FAILED" THEN
            DISPLAY "ALERT: Link ", i + 1, " has failed."

        ELSE IF utilization[i] > 80 THEN
            DISPLAY "ALERT: Link ", i + 1,
                    " is overloaded."

        END IF

    END FOR

    WAIT for a fixed monitoring interval

END WHILE

END
```

Here, the **link quality score** combines bandwidth, delay, and utilization rather than considering only physical distance. A higher bandwidth improves the score, while higher delay and utilization reduce the score. The weights can be adjusted according to the organization's requirements.

The algorithm continuously monitors all links and recalculates the scores at regular intervals. If a link fails or its utilization exceeds **80%**, that link is avoided and an alternate available path is selected. The administrator is also alerted about failures and overloaded links.

This improves **network reliability** because traffic can be moved away from failed links. It improves **routing efficiency** because the selected path is based on current network conditions rather than simply the shortest path. It also provides **fault tolerance**, since alternate paths are automatically considered when the preferred path becomes unavailable. Continuous recalculation allows the network to adapt to changing bandwidth, delay, and utilization throughout the day.